LogMask

Technical Documentation

Reversible, fail-closed pseudonymization of SOC logs

Version 0.27.7

English · Italiano segue

LogMask — Technical Documentation

Version 0.27.7 · Self-hosted, reversible, fail-closed pseudonymization of SOC logs

Table of contents

1. What LogMask is
2. Core concepts
3. Architecture
4. Installation and deployment
5. Configuration
6. User guide
7. Supported formats
8. The pseudonymization engine
9. Token shapes reference
10. Vendor kits
11. DLP categories and Safe mode
12. API reference
13. Security model
14. Operations and troubleshooting
15. Regulatory considerations (EU / Italy, AI use)
16. Extending LogMask

1. What LogMask is

LogMask is a self-hosted web and CLI tool that **pseudonymizes security logs** so that they can be shared — with an external analyst, a vendor support desk, or an AI assistant — without disclosing client-identifying data, while preserving the analytical and forensic value of the log.

The design goal is a specific, asymmetric one: **never leak client-identifying data, even at the cost of readability**, while keeping indicators of compromise (hashes, GUIDs, URLs, file names, ports, event codes) intact so the log remains useful for investigation.

Two properties make this practical:

- **Reversible by design.** Masking is deterministic and reversible on the tenant that produced it: the same value always becomes the same pseudonym, and the holder of the tenant's vault and key can restore the original. This lets an analyst correlate events across a shared export and later map a finding back to the real host or user.
- **Fail-closed.** When LogMask cannot confidently classify a field, it does not pass it through in the clear. In Safe mode the field is elided; unclassified free text that could carry identity is masked as text rather than kept. The default posture is to protect, not to expose.

> **Pseudonymization, not anonymization.** LogMask performs *pseudonymization*, which is reversible by whoever holds the tenant key and vault. It is **not** irreversible anonymization. A subset of transformations is deliberately irreversible — client names (`CLIENT-…`), secrets (`secret-…`) and elided fields (`[ELIDED]`) are never stored and cannot be recovered. The legal consequences of this distinction are covered in §15.

2. Core concepts

Tenant. Every operation happens in the context of a tenant (a client). Each tenant has its own vault, its own derived keys, and its own authorization scope. Two tenants never share pseudonyms: the same host name masked for tenant A and tenant B produces two different, unrelated tokens.

Vault. A per-tenant SQLite database that stores the mapping between a real value and its pseudonym. Original values are stored **encrypted** (AES-GCM, one random nonce per row). The vault is the only thing that makes a masked export reversible; losing it means losing the ability to restore, permanently.

Pseudonym / token. The synthetic value that replaces a real one, e.g. `usr-4ozopszr` for a user or `host-ri6jxfsb.masked.local` for a host. Token shapes are listed in §9.

Kind. The category of a value — user, email, fqdn, ipv4, windomain, iban, and so on. The kind determines the token shape and whether the value is reversible.

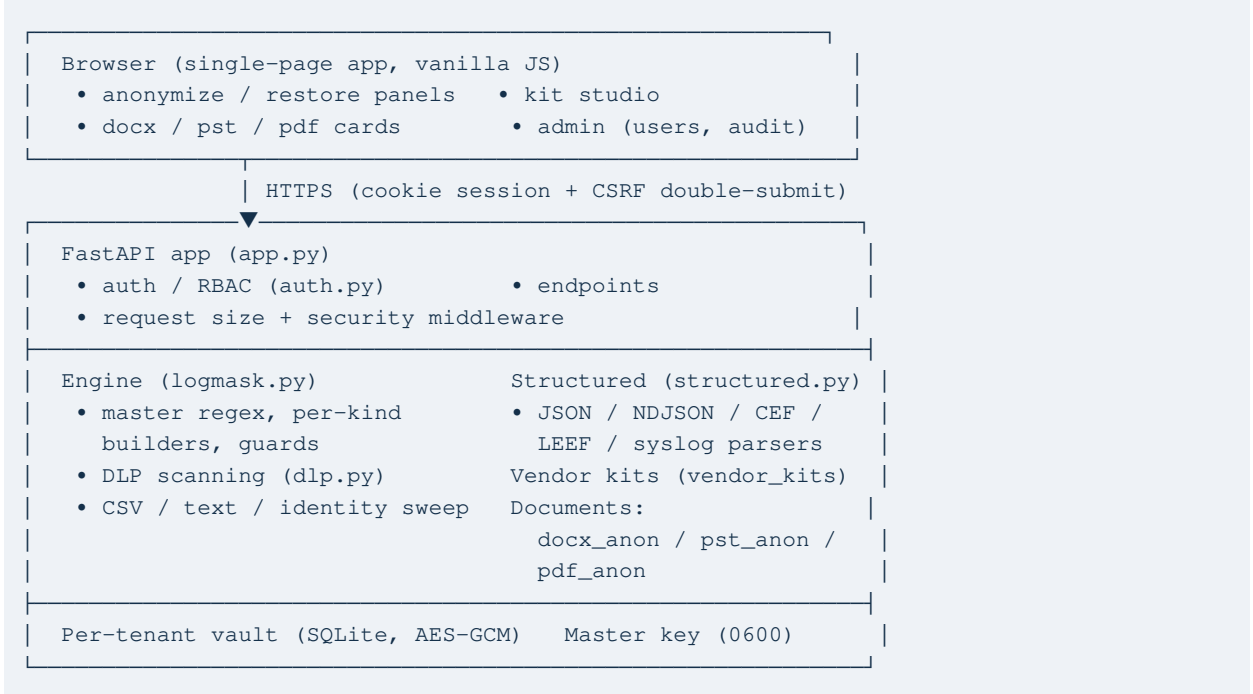
Vendor kit. A YAML rule set that maps a specific product's field names to kinds and actions (mask / keep / text / drop). Kits are what let LogMask classify a Cortex XDR export differently from an Elastic ECS one. See §10.

Safe mode. A fail-closed switch: any field that no kit and no heuristic could classify is elided (`[ELIDED]`) rather than passed through. See §11.

Action. What happens to a field: `mask` (reversible pseudonym), `keep` (left as-is — for operational metadata and IOCs), `text` (free-text scrubbing, for messages and descriptions), `drop` (removed).

3. Architecture

LogMask is a single container running a FastAPI application that serves both the JSON API and a single-page web UI.



Key components:

- `app.py` — HTTP endpoints, authentication dependency, request-size and security-header middleware, upload handling.

- `auth.py` — user store, Argon2 password hashing, session and CSRF token management, RBAC, login rate limiting, audit log.
- `logmask.py` — the pseudonymization engine: the master regex, the per-kind pseudonym builders, all the correctness guards (IOC, opaque-blob, host-original, person-name, VAT/address, identity-in-text), the CSV engine, and the vault-known sweep.
- `structured.py` — format detection and structured parsers (JSON, NDJSON, CEF, LEEF, syslog).
- `dlp.py` — the DLP category catalog and the sensitive-residual scanner (credentials, IBAN, tax id, phone, VAT, address, cloud id, sensitive URL parameters).
- `vendor_kits.py` — kit loading, validation, hot reload, and vendor detection.
- `docx_anon.py`, `pst_anon.py`, `pdf_anon.py` — document handlers for Word, Outlook archives, and PDF.
- `workflows.py` — the preset "workflow profiles" (ticket, AI analysis, threat hunting, report, field quality).

Data at rest: each tenant vault is an encrypted SQLite file under the data directory; the master key is a 32-byte random key stored with `0600` permissions and used to derive per-tenant keys via HMAC.

---

4. Installation and deployment

Requirements

- Docker and Docker Compose.
- No external services: the vault is local SQLite; there is no database server, message broker or cloud dependency.

Quick start

```
# from the project directory
docker compose build --no-cache
docker compose up -d
```

On first start, if no admin password is configured, a random bootstrap password is written to `./data/bootstrap-admin.txt`. Log in as `admin` with that password; you will be required to change it before doing anything else.

The UI is then reachable at `http://<host>:8090/`.

PowerShell (Windows)

```
cd "$env:USERPROFILE\Claude\Projects\logmask-web-v0.10.4"
docker compose down
docker compose build --no-cache
docker compose up -d
```

Network exposure

By default the container binds to `0.0.0.0`, i.e. it is reachable from the whole LAN. For a single-workstation, host-only deployment set:

```
LOGMASK_BIND=127.0.0.1
```

in the `.env` file. For a tool that handles client logs this is worth deciding deliberately; see §13.

Ports

The compose file maps `${LOGMASK_PORT:-8090}:8080` — the app listens on `8080` inside the container, published on `8090` on the host by default.

PDF support and licensing

PDF handling uses **PyMuPDF**, which is distributed under the **AGPL-3.0** license — the same license LogMask itself is released under, so the combined work is license-coherent. Publishing LogMask's source satisfies the AGPL source-availability obligation by construction. For commercial licensing of LogMask without AGPL obligations, see the README (a commercial PyMuPDF license from Artifex is additionally required for the optional PDF module).

---

5. Configuration

Configuration is entirely through environment variables (typically set in `.env`) and optional data files under the data directory. Nothing needs to be hard-coded.

Environment variables

Variable	Default	Purpose
`LOGMASK_DATA`	`/data`	Data directory (vaults, key, config files)
`LOGMASK_BIND`	`0.0.0.0`	Bind address; set `127.0.0.1` for host-only
`LOGMASK_PORT`	`8090`	Published host port
`LOGMASK_ADMIN_USER`	`admin`	Bootstrap admin username
`LOGMASK_ADMIN_PASSWORD`	*(random)*	Bootstrap admin password; if empty, written to `data/bootstrap-admin.txt`
`LOGMASK_KEY_FILE`	`data/master.key`	Master key location (32 bytes, `0600`)
`LOGMASK_COOKIE_SECURE`	`false`	Set `true` behind HTTPS to mark cookies Secure
`LOGMASK_SESSION_IDLE_SECONDS`	`1800`	Idle session timeout
`LOGMASK_SESSION_MAX_SECONDS`	`28800`	Absolute session lifetime
`LOGMASK_LOGIN_MAX_FAILURES`	`5`	Failed logins before lockout
`LOGMASK_LOGIN_WINDOW_SECONDS`	`900`	Lockout window
`LOGMASK_MAX_BODY_BYTES`	`12582912`	Max request body
`LOGMASK_MAX_FILE_BYTES`	`8388608`	Max uploaded file size
`LOGMASK_CLIENT_TERMS` / `_FILE`	—	Client names to mask/scrub (inline or file)
`LOGMASK_CLIENT_TERM_MODE`	`pseudonymize`	`pseudonymize` | `elide` | `label`
`LOGMASK_CLIENT_TERM_LABEL`	`[CLIENTE]`	Label used when mode is `label`
`LOGMASK_HOST_TERMS` / `_FILE`	—	Host-name globs owned by the tenant
`LOGMASK_TENANT_NETWORKS` / `_FILE`	—	Public CIDRs that identify the client
`LOGMASK_PERSON_TERMS` / `_FILE`	—	Real person names present in the tenant
`LOGMASK_KEEP_FIELDS` / `_FILE`	—	Field names to force `keep`
`LOGMASK_DOCX_MAX_UNCOMPRESSED`	`268435456`	Decompressed-size cap for .docx (zip-bomb guard)
`LOGMASK_PST_MAX_EXTRACTED`	`1073741824`	Extracted-size cap for .pst
`LOGMASK_READPST_TIMEOUT`	`300`	Timeout (s) for the readpst subprocess

`LOGMASK_PDF_MAX_PAGES`	`2000`	Page-count cap for PDF

The typical production `.env` raises the upload limits for large XQL/Discover exports, e.g. `LOGMASK_MAX_FILE_BYTES=67108864` and `LOGMASK_MAX_BODY_BYTES=100663296`.

Data files

These live under the data directory and are hot-reloaded where noted. None of them contain real client names in the repository — the repository ships `.example` files only.

File	Purpose
`data/client_terms.txt`	Client names to scrub/mask; one per line
`data/host_terms.txt`	Host-name globs the tenant owns (e.g. `WKS*`, `*DC*`)
`data/tenant_networks.txt`	Public CIDRs that identify the client
`data/person_terms.txt`	Real employee names present in the tenant
`data/keep_fields.txt`	Field names to always keep readable
`data/persons/`	Given-name and surname lists for generic person detection
`data/kits/*.yaml`	User vendor kits (extend or override the built-ins; hot reload)

> **Secret hygiene.** `client_terms.txt` and `person_terms.txt` contain real, identifying values by their nature. They are configuration, not data to be shared, and should be treated with the same care as any secret.

---

6. User guide

The UI is a single page with panels grouped by task.

Anonymize

Paste a log, or upload one or more files, choose the output format (auto-detected by default), and process. The result panel shows the masked output; a report summarizes what was masked, what was kept, what was elided, and — when a vendor kit matched — the detected vendor and coverage.

Key controls:

- **IP policy** — `do not anonymize IPs` / `anonymize internal IPs only` / `anonymize all IPs`. Default: **all**.
- **URL policy** — `do not anonymize URLs` / `customer hosts only` / `mask all URLs`. Default: **all**. Credentials and sensitive query values are always handled regardless of this setting.
- **Safe mode** — elide unclassified fields (fail-closed). Recommended on.
- **Preserve subnet** — keep the /24 grouping of anonymized IPv4s.
- **Mask tenant public networks** — mask the CIDRs in `tenant_networks.txt` even under "internal IPs only".
- **Vendor kit** — force a specific kit, or leave on auto-detect.
- **Workflow profile** — apply a coherent preset (see below).

Restore (de-anonymize)

Paste or upload a masked output and get the original values back. This requires the `reverse` permission and is

recorded in the audit log. Restore works on text, structured formats, CSV, `.docx` and `.pdf`.

Document cards

- **Word .docx** — returns an anonymized `.docx` with layout, tables, headers and numbering preserved; only the text changes. A companion card restores a masked `.docx`.
- **Outlook .pst** — extracts every message and returns one record per message (NDJSON or CSV). The message body is available both as `completeHeader` (the message as it comes out of the archive) and `body` (the same content reduced to human-readable text).
- **PDF** — two modes: `laid-out PDF` (returns a PDF with pages and positions preserved and the original text actually removed) or `text` (extracted, anonymized text). A companion card restores a masked PDF.

Kit studio

Browse the installed kits, open a built-in kit as a starting copy, edit a user kit, validate it, and run a dry-run against a header to see how each column would be classified — without processing any values.

Sessions

Collect multiple logs (paste and/or files) as separate entries, process them together, and download a single ZIP. Sessions live in memory and reset on reload or logout; nothing is written to disk.

Workflow profiles

Presets that apply a coherent set of options. As of 0.26.1 every profile masks all IPs and all URLs by default; the only exception is **Threat hunting (internal)**, which deliberately keeps technical indicators readable for internal correlation.

Profile	Purpose
Customer ticket	Client-facing report: useful technical context, internal data and PII protected
External AI analysis	Cautious preset for external LLMs: everything masked, secrets and PII minimized
Field quality	Anonymization-quality audit: coverage, elisions, field tuning
Threat hunting (internal)	Keeps indicators readable for internal correlation; still blocks secrets and PII
Report / attachment	High-minimization output for attachments, evidence, reports

Administration

User management (create users, assign roles and tenants), the audit log, and the vault reset control (admin only; archives the vault rather than deleting it).

---

7. Supported formats

LogMask detects the format automatically, or you can force it.

Format	Notes
Plain text	Free-text scrubbing across the whole input
CSV / TSV	Per-column classification via kits and heuristics; vertical key/value exports are transposed automatically
JSON / NDJSON	Recursive field masking; arrays and nested objects supported
CEF	ArcSight Common Event Format

LEEF	IBM QRadar Log Event Extended Format
Syslog (key=value)	Structured syslog
Word `.docx`	Layout-preserving; returns a valid `.docx`
Outlook `.pst`	Per-message extraction; requires `pst-utils` in the image
PDF	Layout-preserving redaction or text extraction; requires PyMuPDF

Byte-level robustness: CSV parsing strips embedded NUL bytes (common in Windows event exports) instead of failing; the transport prefixes used by Elastic/Kibana wrappers (`_source.`, `fields.`, `winlog.event_data.`, …) are stripped before field matching so the inner product kit still applies.

---

8. The pseudonymization engine

The masking pass

`Anonymizer.process(text)` runs a master regex over the input that recognizes, in a fixed precedence order: URLs, e-mail addresses, UPNs, IPv4/IPv6, MAC addresses, user-in-path patterns (`C:\Users\jdoe\…`), FQDNs, and Windows domains. Each match is routed to the pseudonym builder for its kind. After the regex pass, a series of additional passes run:

1. **Bare hostnames** — host tokens not caught by the structured patterns.
2. **SharePoint identities** — `/personal/<who>/` paths.
3. **Person names** — first/last name detection using bundled and tenant lists.
4. **Identity-in-prose** — `User mrossi logged on…` narratives (see below).
5. **DLP scan** — credentials, IBAN, tax id, phone, VAT, address, cloud id, sensitive URL parameters.
6. **Client terms** — configured client names, always masked last as a final sweep.

Deterministic, keyed pseudonyms

A pseudonym is a keyed hash of the value:

```
pseudonym = builder( HMAC(tenant_key, kind, normalized_value) )
```

Because the derivation is deterministic and tenant-keyed, the same value always yields the same token within a tenant, different tenants yield different tokens, and the mapping cannot be brute-forced without the tenant key. The original value is stored encrypted in the vault so the token is reversible; irreversible kinds (client name, secret) are never vaulted.

The vault-known sweep

After the main pass, LogMask can sweep the output for values the vault already knows and replace them with their canonical pseudonym — this is how a host named in a free-text message gets the same token it received in its dedicated field. The sweep is cost-adaptive: for a single event against a large vault it searches text→vault via blind index (no decryption); for a huge export against a small vault it reads the vault once. Both strategies produce identical output.

In natural-language contexts (documents, e-mail subjects and bodies) the sweep runs in **prose mode**, which only replaces originals that cannot be common words — those containing a digit or separator (`m.rossi`, `srv-01`, `DOMAIN\user`) or made of multiple words (`Mario Rossi`). This prevents a common word that landed in the vault through a past misclassification (`SOC`, `Windows`, `Sicurezza`) from corrupting readable text.

Correctness guards

Much of the engine is guards that prevent *wrong* masking, which is as damaging as a leak because it destroys IOCs or corrupts text silently:

- **IOC-hash guard** — a run of ≥16 hex characters is an indicator (hash), never a host; it is kept.
- **Opaque-blob guard** — a token adjacent to `+`, `/` or `=` is inside a base64 blob (e.g. an event `_id`) and is not masked, so unique identifiers are not corrupted.
- **Host-original guard** — only values shaped like a machine name are swept; product words (`Windows`, `Management`) and process names (`WmiPrvSE.exe`) are not.
- **Prose-original guard** — in natural language only identifier-shaped originals are swept.
- **Identity-in-text** — key/value identities inside Windows event messages (`Account Name: mrossi`) and serialized JSON (`"SubjectUserName":"mrossi"`) are masked; log placeholders (`-`, `N/A`, `0x0`, `%%1833`, `localhost`, `SYSTEM`) are not.
- **VAT Luhn check** — a bare 11-digit number is masked as an Italian VAT only if it passes the Luhn checksum; otherwise it is treated as a record id and left alone.
- **Address false-positive guard** — un-labeled addresses require a proper-case street name plus a house number, so `Potential RMM Tool Installation via Uncommon Process` and `traffic via proxy 8080` are not mistaken for addresses.

Idempotence and reversibility

Processing an already-masked output does not change it: an existing pseudonym is recognized and left as-is. Restoration reverses every reversible token; irreversible tokens (`CLIENT-…`, `secret-…`, `[ELIDED]`) remain, by design.

---

9. Token shapes reference

Kind	Example input	Example token	Reversible
user	`mrossi`	`usr-4ozopszr`	yes
email	`a@acme.com`	`usr-6rwixc3e@osgwjo.masked`	yes
fqdn / host	`SRV-DC01.corp.local`	`host-ri6jxfsb.masked.local`	yes
ipv4 (internal)	`10.20.30.40`	`198.18.x.x`	yes
ipv4 (public)	`8.8.8.8`	`198.19.x.x`	yes
ipv6	`fe80::1`	`fd00:…::1`	yes
mac	`00:1a:2b:3c:4d:5e`	`02:…` (locally-administered)	yes
windomain	`CORP`	`DOM-shphyawa`	yes
tax id (CF)	`RSSMRA85T10A562S`	`cf-tyvu3zav7vpf`	yes
iban	`IT60X05428…`	`iban-dwchbizxmfec`	yes
phone	`+39 335 1234567`	`tel-vzoftypx33b5`	yes
vat / P.IVA	`00000017897`	`vat-y6orweoro2ds`	yes
address	`Via Roma 12`	`addr-6krvk6xmear5`	yes
person	`Mario Rossi`	`person-… person-…`	yes
cloud id / UUID	`6f0c9a7e-…`	`cloud-g2xiyb2gkiq5`	yes
SID	`S-1-5-21-…`	`S-1-5-21-…` (synthetic)	yes

opaque id	(base64 / hex id)	`id-…`	yes
secret	`password=Estate2024!`	`secret-…`	**no** (never vaulted)
client name	`Acme Spa`	`CLIENT-…`	**no** (never vaulted)
elided	(unclassified, Safe mode)	`[ELIDED]`	**no**

Synthetic IPv4 uses the benchmarking ranges `198.18.0.0/15` (RFC 2544/6890) so it cannot be confused with real production or private addresses; `198.18/16` marks internal sources and `198.19/16` external. Synthetic IPv6 uses `fd00::/8` (unique-local). Synthetic MACs use the locally-administered `02:…` prefix.

---

10. Vendor kits

A kit is a YAML file that maps a vendor's field names to kinds and actions. Detection is by fingerprint fields plus header hints; the highest-scoring kit wins, subject to a minimum confidence.

Built-in kits (21): Acronis, AWS CloudTrail, Bitdefender, Cisco Secure Endpoint, Cortex (Palo Alto XDR / XSIAM / XSOAR), CrowdStrike, Darktrace, Elastic ECS, Exabeam (New-Scale CIM 2.0 / Advanced Analytics / Data Lake), Fortinet, Microsoft Defender, Microsoft Entra, Microsoft Sentinel, Okta, Proofpoint, SentinelOne, Sophos, Splunk CIM, Trend Vision One, Wazuh, Zscaler.

Rule structure. Each rule has a `pattern` (a regex matched against the field name) plus an `action` and, for `mask`, a `kind`:

```
- pattern: ^(dest|src)_user_(sid|dn|ou|entity_id)$
  action: mask
  kind: user
- pattern: ^action_evtlog_event_id$
  action: keep
- pattern: ^(message|raw_log|description)$
  action: text
```

Rules are evaluated top to bottom; specific rules must precede generic catch-alls (a field like `action_evtlog_event_id` must be kept *before* the generic `.*_id$` rule would mask it).

User kits. Files in `data/kits/*.yaml` are loaded on top of the built-ins with hot reload and take priority. Use the kit studio to open a built-in kit as a copy, edit, validate, and dry-run it against a header before saving.

Coverage is measured, not assumed. The Exabeam kit, for example, is built on the 1165 field names published in the official `ExabeamLabs/CIMLibrary` repository; a regression test verifies that no field whose name denotes an identity is left readable, while operational fields stay readable.

No kit may keep unknown fields in the clear. A vendor kit rule is tagged `vendor:`, and Safe mode deliberately does not re-elide vendor-classified fields — so a catch-all `.* → keep` at the end of a kit would keep *every* unrecognized field readable, defeating fail-closed. A class-wide regression test forbids a `.* → keep` (and a `.* → mask` with a non-masking kind) in any kit. Where a kit needs a catch-all for a field-dense vendor, it uses `.* → text`, which processes unknown values as free text — masking any identity inside them (IP, e-mail, host, `DOMAIN\user`, person or client name) while leaving operational content readable. Every kit must classify users and IPs, and hosts too except for vendors that have no host concept (Proofpoint, AWS CloudTrail, Okta, Entra), which are listed explicitly so an exception is a decision, not an oversight.

---

11. DLP categories and Safe mode

Independently of vendor kits, a DLP scan finds high-confidence sensitive values anywhere in the output —

including inside free text. Each category has a default action and can be overridden per request.

Category	Kind	Default
`credentials`	secret	redact
`private_key`	secret	redact
`tax_id`	taxid	pseudonymize
`iban`	iban	pseudonymize
`phone`	phone	pseudonymize
`person_name`	person	pseudonymize
`address`	address	pseudonymize
`vat_id`	vat	pseudonymize
`cloud_id`	cloud	pseudonymize
`sensitive_url`	secret	redact

Actions: `pseudonymize` (reversible token), `redact` (`[ELIDED]`, irreversible), `block` (fail the output), `keep` (leave in the clear).

Safe mode is the fail-closed switch. With Safe mode on, any populated field that no kit and no heuristic could classify is elided. The rationale is that an unclassified field is exactly where an unexpected identifier hides; passing it through in the clear would be the one place a leak slips out.

Documents and mail never elide. On the `.docx`, `.pst` and `.pdf` paths, eliding a field is a net loss — the returned file loses text and restore cannot rebuild it. On those paths every sensitive value becomes a pseudonym instead of `[ELIDED]`. Secrets are the one exception on those paths: they become a deterministic `secret-…` token that is never written to the vault, so the document stays readable but the secret is not recoverable and the tool never becomes a store of clients' passwords.

Untracked-field overrides

Fields that no kit classifies are elided in Safe mode. To change how a specific field is treated without writing a kit rule, the report shows each untracked field with a dropdown offering three choices, and a **Save missing-field config** button:

- **keep** — leave the value readable (operational metadata, IOCs);
- **pseudonymize** — reversible `id-…` token (always the generic `opaque` kind, which masks any value; a typed kind such as `ipv4` is deliberately not offered because it would return non-conforming values unchanged — a leak);
- **elide** — `[ELIDED]`.

The choices are stored globally by field name in `data/field_overrides.json` (hot reload) and apply to every format — CSV, JSON/NDJSON, CEF, LEEF, syslog — because they are resolved before vendor detection. They are independent of the detected vendor, since untracked fields usually appear precisely when no kit matched. Overrides win over kits, catalog and heuristics, but a field with no override still elides in Safe mode: fail-closed is preserved. Saving requires the `admin` permission, like other configuration changes. Since 0.27.0 `redact` is also a valid kit action, so a user kit can force elision of a field too.

---

12. API reference

All endpoints are under `/api`. Session is a cookie; state-changing requests require the CSRF token (double-

submit: the `logmask_csrf` cookie value echoed in the `X-CSRF-Token` header). Every request is authorized by role and, where relevant, by tenant.

Authentication

Method	Path	Permission	Purpose
POST	`/api/login`	—	Log in; returns session + CSRF cookies
GET	`/api/me`	session	Current user, roles, capabilities
POST	`/api/logout`	session	End session
POST	`/api/change-password`	session	Change own password

Anonymize / restore

Method	Path	Permission	Purpose
POST	`/api/anonymize`	anonymize	Text / CSV / structured input (JSON body)
POST	`/api/anonymize-docx`	anonymize	Word document (multipart)
POST	`/api/anonymize-pst`	anonymize	Outlook archive (multipart)
POST	`/api/anonymize-pdf`	anonymize	PDF (multipart; `output=pdf|text`)
POST	`/api/deanonymize`	reverse	Restore text / structured / CSV
POST	`/api/deanonymize-docx`	reverse	Restore a masked `.docx`
POST	`/api/deanonymize-pdf`	reverse	Restore a masked PDF

Kits and policy

Method	Path	Permission	Purpose
GET	`/api/vendor-kits`	session	List installed kits
POST	`/api/kit-dry-run`	session	Classify a header without processing values
GET	`/api/kits/files`	admin	List user kit files
GET	`/api/kits/files/{name}`	admin	Read a user kit
PUT	`/api/kits/files/{name}`	admin	Create/update a user kit (validated)
DELETE	`/api/kits/files/{name}`	admin	Delete a user kit
POST	`/api/kits/validate`	admin	Validate kit YAML
GET	`/api/kits/bundled/{kit_id}`	admin	Read a built-in kit
GET	`/api/dlp-categories`	session	DLP category catalog
GET	`/api/field-overrides`	session	Current per-field overrides
POST	`/api/field-overrides`	admin	Save per-field overrides (keep/mask/redact)
GET	`/api/workflow-profiles`	session	Workflow presets

Reporting and admin

Method	Path	Permission	Purpose
GET	`/api/tenants`	session	Tenants the user may access
GET	`/api/fields`	reports	Field-level statistics

GET	`/api/stats`	reports	Vault statistics
GET	`/api/admin/users`	admin	List users
POST	`/api/admin/users`	admin	Create a user
POST	`/api/admin/vault/reset`	admin	Archive-and-reset a tenant vault
POST	`/api/admin/secret/reset`	admin	Regenerate the master key (archives key + all vaults)
GET	`/api/admin/audit`	audit	Audit log

Example — anonymize text

```
curl -X POST http://localhost:8090/api/anonymize \
  -H "Content-Type: application/json" \
  -H "X-CSRF-Token: <csrf>" \
  --cookie "logmask_session=<session>; logmask_csrf=<csrf>" \
  -d '{"tenant":"acme","text":"user=mrossi src=10.20.30.40","format":"auto",
       "safe_mode":true,"ip_mode":"all","url_mode":"all"}'
```

Errors are returned as JSON with a `detail` field and, for unexpected errors, an `error_type`; the full traceback stays in the container logs and is never exposed.

---

13. Security model

Authentication and sessions

- **Passwords** are hashed with Argon2 (`time_cost=3`, `memory_cost=64 MiB`, `parallelism=2`); minimum length 12; the username may not appear in the password.
- **Sessions** are cookies marked `HttpOnly` and `SameSite=strict`; set `LOGMASK_COOKIE_SECURE=true` behind HTTPS. Idle and absolute lifetimes are enforced.
- **CSRF** uses double-submit: a non-`HttpOnly` `logmask_csrf` cookie must be echoed in the `X-CSRF-Token` header on every state-changing request.
- **Login rate limiting**: after `LOGMASK_LOGIN_MAX_FAILURES` failures within the window, the account is locked for the window duration.
- **Bootstrap admin** must change its password on first login before any other action is permitted.

Roles (RBAC)

Role	Permissions
`operator`	anonymize
`analyst`	anonymize, reports
`reverser`	anonymize, reports, reverse
`admin`	anonymize, reports, reverse, admin, audit

Restoration (`reverse`) is a distinct, higher privilege and every reversal is written to the audit log with user, tenant, IP and outcome.

Tenant isolation

Tenant identifiers are validated by regex and canonicalized to a path-safe form (no `./..` traversal), then authorized against the user's tenant scope. Vaults and derived keys are per tenant; a user can only operate on

tenants they are granted.

Cryptography

- A 32-byte master key stored with `0600` permissions.
- Per-tenant keys derived via HMAC.
- Original values encrypted with AES-GCM, one random 12-byte nonce per row.
- Blind indexes (keyed hashes) let the sweep look values up without decrypting the vault.

Input hardening

- **Request and file size limits** (413 on exceed).
- **`.docx` zip-bomb guard**: a decompressed-size cap with a double check (declared sizes, then a read budget), because the zip index can lie. Verified: a 200 KB document that expands to hundreds of MB is rejected in milliseconds.
- **`.pst` extraction cap**: the extracted size is bounded; `readpst` runs with a timeout and closed stdin so a corrupt or password-protected archive cannot hang the worker.
- **PDF**: page-count cap; encrypted/corrupt PDFs are rejected with a clear message; the produced PDF is re-read and verified that no original value is still extractable before delivery.
- **XML entity expansion** (billion-laughs) is blocked by the parser.
- **CSV NUL bytes** are stripped rather than crashing the parser.
- **Content-Security-Policy** and `Cache-Control: no-store` headers are set; all dynamic DOM in the UI is inserted via `textContent`/escaping, not raw HTML.

Residual considerations (operator responsibility)

- The default bind is `0.0.0.0` (LAN-reachable); set `LOGMASK_BIND=127.0.0.1` for host-only use.
- The container runs as root; the input caps above mitigate but do not eliminate this.
- `readpst` is third-party C code; the timeout and size cap bound its impact, not its internal vulnerabilities.
- `client_terms.txt` and `person_terms.txt` hold real identifying values and must be protected accordingly.

---

14. Operations and troubleshooting

Vault reset

`Reset vault` (admin only) archives the tenant vault with a timestamp rather than deleting it. Because the vault is the only thing that makes past exports reversible, resetting means losing the mapping for everything already shared; archiving allows recovery if the reset was a mistake. Permanent deletion remains a deliberate, manual choice.

Reset secret (master key)

`Reset secret` (admin only) regenerates the master key — the secret from which **all** tokens derive. A confirmation pop-up summarizes the risk and requires typing the word `RESET`. The effect is global and destructive to reversibility:

- every future token changes (the same value yields a different pseudonym, `secret-…` included);
- **every existing vault, of every tenant, stops being reversible**, because it was encrypted with the previous key.

As with the vault reset, nothing is deleted: the old key and all existing vaults are **archived** with a timestamp (`master-prereset-…key`, `vault-prereset-…db`) and are recoverable **only together** — restoring both puts you back. Users and sessions do not depend on the master key, so nobody is logged out, and the change takes effect live without a restart.

Use this to obtain a fresh, disconnected secret — for example after copying the folder to another machine, if you deliberately want the new deployment to produce different tokens. Conversely, to preserve continuity across machines, copy `data/master.key` (and the vaults) rather than resetting.

Audit log

Every anonymize, reverse, field-override save and secret-reset operation is recorded with action, user, tenant, IP, and outcome (plus counts and, for failures, the exception type — never the log content). Use it to review who restored or reset what and when.

Common situations

Symptom	Likely cause / action
`PST anonymization failed: Failed to fetch`	The `.pst` was still open in Outlook; the file changed during upload. Close Outlook or work on a copy. LogMask reads the file into a stable copy before sending.
`il PDF è protetto da password`	Remove the PDF password (open and save a copy without protection) before anonymizing.
A field comes out in the clear	It was not classified by any kit; enable Safe mode, or add a user kit rule / a `keep_fields` entry, or run a kit dry-run to see the classification.
Everything is `[ELIDED]`	No vendor kit matched and Safe mode elided the unknown fields; force the correct kit or add a user kit.
Restore leaves some values as pseudonyms	Those tokens are irreversible by design (`CLIENT-…`, `secret-…`) or belong to a different tenant's vault.
It got slow	An earlier release had a vault-size-dependent sweep; current releases are vault-independent. Very large single tokens were a ReDoS vector fixed in 0.26.2.

Upgrading

Copy the new release over the project folder (or extract the release ZIP), then `docker compose down && docker compose build --no-cache && docker compose up -d`. Vaults and configuration under the data directory are preserved across upgrades.

---

15. Regulatory considerations (EU / Italy, AI use)

> **This section is informational, not legal advice.** It summarizes how LogMask relates to the EU and Italian legal framework as of 2026 so that you can make an informed assessment with your DPO or counsel. It is not a compliance opinion, and LogMask's authors are not your lawyers.

15.1 The central legal fact: pseudonymization is not anonymization

Under the GDPR (Regulation (EU) 2016/679), **pseudonymization** is defined in Article 4(5) as processing personal data so that it can no longer be attributed to a data subject without additional information kept separately. Crucially, **Recital 26 states that pseudonymized data which could be attributed to a natural person by the use of additional information is still personal data.** Anonymization, by contrast, is irreversible and — per Recital 26 — falls outside the GDPR.

LogMask performs pseudonymization. Therefore:

- **A LogMask-masked export is, in general, still personal data** under the GDPR, because the vault plus the tenant key is exactly the "additional information" that allows re-identification. Processing it (storing, sharing, sending to an AI service) still requires a legal basis and still triggers GDPR obligations.
- **The vault and master key are the "additional information"** that Article 4(5) requires be "kept separately and subject to technical and organizational measures." LogMask's per-tenant AES-GCM encryption, `0600` master key, RBAC and audit log are the kind of measures the article contemplates — but the *separation* (keeping the vault out of the recipient's hands) is an operational responsibility: never ship the vault together with a masked export.
- **The irreversible transformations behave differently.** Values that LogMask never stores — client names (`CLIENT-…`), secrets (`secret-…`) and elided fields (`[ELIDED]`) — are not reversible even by the vault holder, and for those specific values the export approaches anonymization. The rest remains pseudonymous.

The practical upshot: LogMask is a **data-minimization and risk-reduction control**, not a magic wand that removes data from GDPR scope. It substantially reduces the personal data exposed to a recipient and supports the minimization principle (Art. 5(1)(c)) and security-of-processing obligation (Art. 32), which is precisely its value.

15.2 Sending logs to an AI service

The specific use case LogMask targets — masking a log before sending it to an external LLM or AI assistant — sits inside two overlapping regimes.

GDPR. Sending a log to a third-party AI service is a processing operation and, if the provider is outside the EU/EEA, potentially a transfer. It needs a legal basis, a data-processing agreement with the provider, and adherence to the minimization principle. The Italian supervisory authority (**Garante per la protezione dei dati personali**) has repeatedly stressed the difficulty of applying minimization to generative-AI services and has taken enforcement action in this area (notably the ChatGPT case, concluded in December 2024 with a €15 million sanction; note that an Italian court later annulled the related measure, so the case law is still evolving). Pseudonymizing the log before it leaves your control is a direct, defensible way to honor minimization: the AI service sees `usr-4ozopszr`, not `mrossi`.

EU AI Act (Regulation (EU) 2024/1689). The AI Act applies progressively. Obligations for general-purpose AI models have applied since **2 August 2025**; a major milestone falls on **2 August 2026**, when transparency obligations (Art. 50, e.g. disclosing that a user is interacting with an AI system) and the sanctions regime for GPAI take effect (fines up to 3% of worldwide annual turnover or €15 million, whichever is higher). Timelines for some high-risk systems were subsequently deferred (to December 2027 and August 2028) by the "Digital Omnibus." LogMask itself is not an AI system and is not regulated by the AI Act; its relevance is that it is a control you can place *in front of* an AI system to reduce the personal data that system processes.

15.3 The Italian national AI law (Legge 132/2025)

Italy enacted its first organic AI law, **Law No. 132 of 23 September 2025** ("Provisions and delegations to the Government on artificial intelligence"), published in the Official Gazette on 25 September 2025 and **in force since 10 October 2025**. It is expressly coordinated with the EU AI Act and adds national-level direction. It designates two national authorities: the **Agency for Digital Italy (AgID)** for promotion and development, and the **National Cybersecurity Agency (ACN)** for supervision and security. For an organization using AI on data that may contain personal or security-sensitive information, the law reinforces principles of transparency, human oversight and security that a pre-processing control like LogMask helps operationalize.

15.4 What LogMask does and does not give you

It helps you: minimize the personal and client-identifying data exposed when a log is shared or sent to an AI service; keep a reversible, auditable mapping under your own control; demonstrate a concrete technical measure toward Art. 5(1)(c) minimization and Art. 32 security; and irreversibly remove the most sensitive items (client names, secrets).

It does not, by itself: make a masked export "anonymous" or remove it from GDPR scope (the vault makes it re-identifiable); provide a legal basis for processing or transfer; replace a DPA with an AI provider; or absolve you of a DPIA where one is required. Those remain organizational decisions.

Operational guidance that follows from the above:

1. **Never ship the vault with the export.** Separation is what keeps the pseudonymization meaningful under Art. 4(5).
2. **Protect the vault and master key** as the sensitive assets they are — they are the re-identification key.
3. **Prefer the more protective settings** when the recipient is external, especially an AI service: all IPs and URLs masked (the 0.26.1 defaults), Safe mode on, secrets and PII minimized. The "External AI analysis" workflow profile encodes this.
4. **Treat** `client_terms.txt` / `person_terms.txt` as confidential configuration.
5. **Keep the audit log** as evidence of who reversed what and when.

---

16. Extending LogMask

Add or override a vendor kit

Create a YAML file in `data/kits/`. It is hot-reloaded and takes priority over the built-in of the same id. Use the kit studio to start from a built-in copy, then validate and dry-run before saving. Order specific rules before generic catch-alls, and prefer `keep` for operational fields and IOCs, `mask` (with a `kind`) for identities, and `text` for free-text fields.

Add a DLP detector

DLP categories live in `dlp.py` with a label, description, default action, and kind. A new detector adds a regex to the residual scanner and, where the value should be reversible, a matching pseudonym builder kind in `logmask.py`. New categories are surfaced automatically in the UI DLP panel.

Tune person / host / client detection

Populate the data files: `person_terms.txt` for real employees (masked even as a bare token), `host_terms.txt` for the tenant's host-name globs, `tenant_networks.txt` for public CIDRs that identify the client, `client_terms.txt` for client names. The bundled `data/persons/` lists drive generic name detection.

Test discipline

Every behavioral change ships with a regression test; the suite is the specification of intended behavior, especially the guards (a masking that is too aggressive is treated as a bug on par with a leak). Run the full suite before building a release.

LogMask 0.27.7 — technical documentation. For the change history see `CHANGELOG.md`.

LogMask

Documentazione Tecnica

Pseudonimizzazione reversibile e fail-closed di log SOC

Version 0.27.7

Italiano

LogMask — Documentazione tecnica

Versione 0.27.7 · Pseudonimizzazione reversibile e fail-closed di log SOC, self-hosted

---

Indice

1. Cos'è LogMask
2. Concetti fondamentali
3. Architettura
4. Installazione e deployment
5. Configurazione
6. Guida all'uso
7. Formati supportati
8. Il motore di pseudonimizzazione
9. Riferimento delle forme dei token
10. Kit vendor
11. Categorie DLP e Safe mode
12. Riferimento API
13. Modello di sicurezza
14. Operazioni e risoluzione problemi
15. Considerazioni normative (UE / Italia, uso dell'AI)
16. Estendere LogMask

---

1. Cos'è LogMask

LogMask è uno strumento self-hosted, con interfaccia web e CLI, che **pseudonimizza i log di sicurezza** così che possano essere condivisi — con un analista esterno, il supporto di un vendor o un assistente AI — senza rivelare i dati che identificano il cliente, preservando al tempo stesso il valore analitico e forense del log.

L'obiettivo di progetto è specifico e asimmetrico: **non far mai trapelare dati che identificano il cliente, anche a costo della leggibilità**, mantenendo intatti gli indicatori di compromissione (hash, GUID, URL, nomi di file, porte, event code) perché il log resti utile all'indagine.

Due proprietà lo rendono praticabile:

- **Reversibile per costruzione.** Il mascheramento è deterministico e reversibile sul tenant che l'ha prodotto: lo stesso valore diventa sempre lo stesso pseudonimo, e chi possiede il vault e la chiave del tenant può ripristinare l'originale. Questo permette a un analista di correlare gli eventi in un export condiviso e, in seguito, di risalire da un riscontro all'host o all'utente reale.
- **Fail-closed.** Quando LogMask non riesce a classificare con sicurezza un campo, non lo lascia passare in chiaro. In Safe mode il campo viene eliso; il testo libero non classificato che potrebbe contenere un'identità viene mascherato come testo invece che tenuto. La postura predefinita è proteggere, non esporre.

> **Pseudonimizzazione, non anonimizzazione.** LogMask esegue *pseudonimizzazione*, reversibile da chi possiede chiave e vault del tenant. **Non** è anonimizzazione irreversibile. Un sottoinsieme di trasformazioni è volutamente irreversibile — i nomi cliente (`CLIENT-…`), i segreti (`secret-…`) e i campi elisi

> ([ELIDED]) non vengono mai memorizzati e non sono recuperabili. Le conseguenze giuridiche di questa distinzione sono trattate nel §15.

---

2. Concetti fondamentali

Tenant. Ogni operazione avviene nel contesto di un tenant (un cliente). Ogni tenant ha il proprio vault, le proprie chiavi derivate e il proprio ambito di autorizzazione. Due tenant non condividono mai gli pseudonimi: lo stesso nome host mascherato per il tenant A e per il tenant B produce due token diversi e slegati.

Vault. Un database SQLite per tenant che conserva la corrispondenza tra un valore reale e il suo pseudonimo. I valori originali sono conservati **cifrati** (AES-GCM, un nonce casuale per riga). Il vault è l'unica cosa che rende reversibile un export mascherato; perderlo significa perdere per sempre la capacità di ripristinare.

Pseudonimo / token. Il valore sintetico che sostituisce quello reale, es. `usr-4ozopszr` per un utente o `host-ri6jxfsb.masked.local` per un host. Le forme dei token sono nel §9.

Kind (tipo). La categoria di un valore — user, email, fqdn, ipv4, windomain, iban e così via. Il tipo determina la forma del token e se il valore è reversibile.

Kit vendor. Un insieme di regole YAML che mappa i nomi campo di un prodotto specifico a tipi e azioni (mask / keep / text / drop). I kit sono ciò che permette a LogMask di classificare un export Cortex XDR diversamente da uno Elastic ECS. Vedi §10.

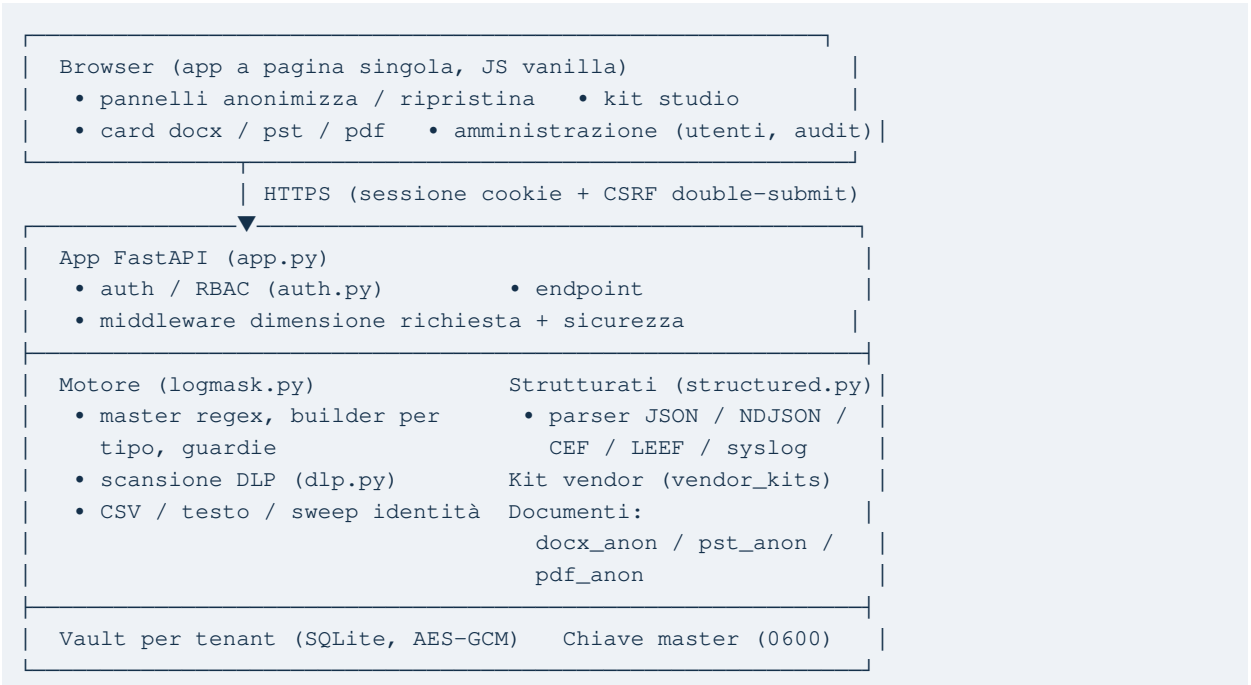
Safe mode. Un interruttore fail-closed: qualsiasi campo che nessun kit e nessuna euristica ha saputo classificare viene eliso ([ELIDED]) invece di passare in chiaro. Vedi §11.

Azione. Cosa accade a un campo: `mask` (pseudonimo reversibile), `keep` (lasciato com'è — per metadati operativi e IOC), `text` (scrubbing di testo libero, per messaggi e descrizioni), `drop` (rimosso).

---

3. Architettura

LogMask è un singolo container che esegue un'applicazione FastAPI, la quale serve sia l'API JSON sia l'interfaccia web a pagina singola.



Componenti principali:

- `app.py` — endpoint HTTP, dipendenza di autenticazione, middleware per dimensione della richiesta e header di sicurezza, gestione degli upload.
- `auth.py` — archivio utenti, hashing password Argon2, gestione sessioni e token CSRF, RBAC, rate limiting sul login, log di audit.
- `logmask.py` — il motore di pseudonimizzazione: la master regex, i builder di pseudonimi per tipo, tutte le guardie di correttezza (IOC, blob opaco, host-originale, nomi di persona, P.IVA/indirizzi, identità nel testo), il motore CSV e lo sweep dei valori noti al vault.
- `structured.py` — rilevamento del formato e parser strutturati (JSON, NDJSON, CEF, LEEF, syslog).
- `dlp.py` — il catalogo delle categorie DLP e lo scanner dei residui sensibili (credenziali, IBAN, codice fiscale, telefono, P.IVA, indirizzo, id cloud, parametri URL sensibili).
- `vendor_kits.py` — caricamento kit, validazione, hot reload e rilevamento vendor.
- `docx_anon.py`, `pst_anon.py`, `pdf_anon.py` — gestori documentali per Word, archivi Outlook e PDF.
- `workflows.py` — i "profili di lavoro" preimpostati (ticket, analisi AI, threat hunting, report, qualità campi).

Dati a riposo: ogni vault di tenant è un file SQLite cifrato nella cartella dati; la chiave master è una chiave casuale di 32 byte con permessi `0600`, usata per derivare le chiavi per tenant tramite HMAC.

---

4. Installazione e deployment

Requisiti

- Docker e Docker Compose.
- Nessun servizio esterno: il vault è SQLite locale; non c'è un server di database, un broker di messaggi o una dipendenza cloud.

Avvio rapido

```
# dalla cartella del progetto
docker compose build --no-cache
docker compose up -d
```

Al primo avvio, se non è configurata alcuna password admin, una password di bootstrap casuale viene scritta in `./data/bootstrap-admin.txt`. Accedi come `admin` con quella password; ti sarà richiesto di cambiarla prima di ogni altra operazione.

L'interfaccia è poi raggiungibile su `http://<host>:8090/`.

PowerShell (Windows)

```
cd "$env:USERPROFILE\Claude\Projects\logmask-web-v0.10.4"
docker compose down
docker compose build --no-cache
docker compose up -d
```

Esposizione di rete

Per impostazione predefinita il container si lega a `0.0.0.0`, cioè è raggiungibile da tutta la LAN. Per un deployment su singola postazione, accessibile solo dall'host, imposta:

```
LOGMASK_BIND=127.0.0.1
```

nel file `.env`. Per uno strumento che tratta log di clienti vale la pena deciderlo consapevolmente; vedi §13.

Porte

Il file compose mappa `${LOGMASK_PORT:-8090}:8080` — l'app ascolta su `8080` dentro il container, pubblicata di default su `8090` sull'host.

Supporto PDF e licenza

La gestione dei PDF usa **PyMuPDF**, distribuito con licenza **AGPL-3.0** — la stessa con cui è rilasciato LogMask, quindi l'opera combinata è coerente sul piano delle licenze. La pubblicazione del sorgente di LogMask soddisfa per costruzione l'obbligo AGPL di disponibilità del sorgente. Per il licenziamento commerciale di LogMask senza obblighi AGPL vedi il README (per il modulo PDF opzionale serve in aggiunta una licenza commerciale PyMuPDF da Artifex).

---

5. Configurazione

La configurazione avviene interamente tramite variabili d'ambiente (tipicamente nel `.env`) e file dati opzionali nella cartella dati. Nulla va scritto nel codice.

Variabili d'ambiente

Variabile	Default	Scopo
`LOGMASK_DATA`	`/data`	Cartella dati (vault, chiave, file di config)
`LOGMASK_BIND`	`0.0.0.0`	Indirizzo di bind; `127.0.0.1` per solo host
`LOGMASK_PORT`	`8090`	Porta pubblicata sull'host
`LOGMASK_ADMIN_USER`	`admin`	Username admin di bootstrap
`LOGMASK_ADMIN_PASSWORD`	*(casuale)*	Password admin di bootstrap; se vuota, scritta in `data/bootstrap-admin.txt`
`LOGMASK_KEY_FILE`	`data/master.key`	Posizione chiave master (32 byte, `0600`)
`LOGMASK_COOKIE_SECURE`	`false`	`true` dietro HTTPS per marcare i cookie Secure
`LOGMASK_SESSION_IDLE_SECONDS`	`1800`	Timeout sessione inattiva
`LOGMASK_SESSION_MAX_SECONDS`	`28800`	Durata massima assoluta della sessione
`LOGMASK_LOGIN_MAX_FAILURES`	`5`	Login falliti prima del blocco
`LOGMASK_LOGIN_WINDOW_SECONDS`	`900`	Finestra di blocco
`LOGMASK_MAX_BODY_BYTES`	`12582912`	Corpo massimo della richiesta
`LOGMASK_MAX_FILE_BYTES`	`8388608`	Dimensione massima file caricato
`LOGMASK_CLIENT_TERMS` / `_FILE`	—	Nomi cliente da mascherare/scrubbare (inline o file)
`LOGMASK_CLIENT_TERM_MODE`	`pseudonymize`	`pseudonymize` | `elide` | `label`
`LOGMASK_CLIENT_TERM_LABEL`	`[CLIENTE]`	Etichetta usata quando la modalità è `label`
`LOGMASK_HOST_TERMS` / `_FILE`	—	Glob di nomi host di proprietà del tenant
`LOGMASK_TENANT_NETWORKS` / `_FILE`	—	CIDR pubblici che identificano il cliente
`LOGMASK_PERSON_TERMS` / `_FILE`	—	Nomi di persone realmente presenti nel tenant
`LOGMASK_KEEP_FIELDS` / `_FILE`	—	Nomi campo da forzare a `keep`

`LOGMASK_DOCX_MAX_UNCOMPRESSED`	`268435456`	Tetto sul decompresso .docx (guardia zip-bomb)
`LOGMASK_PST_MAX_EXTRACTED`	`1073741824`	Tetto sull'estratto .pst
`LOGMASK_READPST_TIMEOUT`	`300`	Timeout (s) del sottoprocesso readpst
`LOGMASK_PDF_MAX_PAGES`	`2000`	Tetto sul numero di pagine PDF

Il tipico `.env` di produzione alza i limiti di upload per gli export XQL/Discover grandi, es. `LOGMASK_MAX_FILE_BYTES=67108864` e `LOGMASK_MAX_BODY_BYTES=100663296`.

File dati

Stanno nella cartella dati e sono ricaricati a caldo dove indicato. Nessuno di essi contiene nomi cliente reali nel repository — il repository include solo file `.example`.

File	Scopo
`data/client_terms.txt`	Nomi cliente da scrubbare/mascherare; uno per riga
`data/host_terms.txt`	Glob di nomi host di proprietà del tenant (es. `WKS*`, `*DC*`)
`data/tenant_networks.txt`	CIDR pubblici che identificano il cliente
`data/person_terms.txt`	Nomi di dipendenti reali presenti nel tenant
`data/keep_fields.txt`	Nomi campo da tenere sempre leggibili
`data/persons/`	Liste di nomi e cognomi per il rilevamento generico di persone
`data/kits/*.yaml`	Kit vendor utente (estendono o sostituiscono quelli di serie; hot reload)

> **Igiene dei segreti.** `client_terms.txt` e `person_terms.txt` contengono per natura valori reali e identificativi. Sono configurazione, non dati da condividere, e vanno trattati con la stessa cura di un segreto.

---

6. Guida all'uso

L'interfaccia è una pagina unica con pannelli raggruppati per attività.

Anonimizza

Incolla un log, oppure carica uno o più file, scegli il formato di output (rilevato automaticamente di default) ed elabora. Il pannello risultato mostra l'output mascherato; un report riassume cosa è stato mascherato, cosa tenuto, cosa eliso e — quando un kit vendor ha fatto match — il vendor rilevato e la copertura.

Controlli principali:

- **Policy IP** — `non anonimizzare IP` / `solo IP interni` / `tutti gli IP`. Default: **tutti**.
- **Policy URL** — `non anonimizzare URL` / `solo host del cliente` / `tutti gli URL`. Default: **tutti**. Le credenziali e i valori di query sensibili vengono comunque trattati, indipendentemente da questa scelta.
- **Safe mode** — elide i campi non classificati (fail-closed). Consigliata attiva.
- **Preserva subnet** — mantiene il raggruppamento /24 degli IPv4 anonimizzati.
- **Maschera reti pubbliche del tenant** — maschera i CIDR di `tenant_networks.txt` anche con "solo IP interni".
- **Kit vendor** — forza un kit specifico, oppure lascia il rilevamento automatico.
- **Profilo di lavoro** — applica un preset coerente (vedi sotto).

Ripristina (de-anonimizza)

Incolla o carica un output mascherato e riottieni i valori originali. Richiede il permesso `reverse` ed è registrato nel log di audit. Il ripristino funziona su testo, formati strutturati, CSV, `.docx` e `.pdf`.

Card documenti

- **Word .docx** — restituisce un `.docx` anonimizzato con stili, tabelle, intestazioni e numerazione preservati; cambia solo il testo. Una card gemella ripristina un `.docx` mascherato.
- **Outlook .pst** — estrae ogni messaggio e restituisce un record per messaggio (NDJSON o CSV). Il corpo del messaggio è disponibile sia come `completeHeader` (il messaggio come esce dall'archivio) sia come `body` (lo stesso contenuto ridotto a testo leggibile).
- **PDF** — due modalità: `PDF impaginato` (restituisce un PDF con pagine e posizioni preservate e il testo originale realmente rimosso) oppure `testo` (testo estratto e anonimizzato). Una card gemella ripristina un PDF mascherato.

Kit studio

Sfoglia i kit installati, apri un kit di serie come copia di partenza, modifica un kit utente, validalo ed esegui un dry-run su un'intestazione per vedere come verrebbe classificata ogni colonna — senza elaborare alcun valore.

Sessioni

Raccogli più log (incolla e/o file) come voci separate, elaborale insieme e scarica un unico ZIP. Le sessioni vivono in memoria e si azzerano al reload o al logout; nulla viene scritto su disco.

Profili di lavoro

Preset che applicano un insieme coerente di opzioni. Dalla 0.26.1 ogni profilo maschera per default tutti gli IP e tutti gli URL; l'unica eccezione è **Threat hunting (interno)**, che volutamente mantiene leggibili gli indicatori tecnici per la correlazione interna.

Profilo	Scopo
Ticket cliente	Report verso il cliente: contesto tecnico utile, dati interni e PII protetti
Analisi AI esterna	Preset prudente per LLM esterni: tutto mascherato, segreti e PII minimizzati
Qualità campi	Audit della qualità di anonimizzazione: copertura, elisioni, tuning campi
Threat hunting (interno)	Mantiene leggibili gli indicatori per la correlazione interna; blocca comunque segreti e PII
Report / allegato	Output ad alta minimizzazione per allegati, evidenze, report

Amministrazione

Gestione utenti (creazione, assegnazione di ruoli e tenant), log di audit e controllo di reset del vault (solo admin; archivia il vault invece di eliminarlo).

---

7. Formati supportati

LogMask rileva il formato automaticamente, oppure puoi forzarlo.

Formato	Note
Testo semplice	Scrubbing di testo libero sull'intero input

CSV / TSV	Classificazione per colonna tramite kit ed euristiche; gli export verticali chiave/valore vengono ribaltati automaticamente
JSON / NDJSON	Mascheramento ricorsivo dei campi; array e oggetti annidati supportati
CEF	ArcSight Common Event Format
LEEF	IBM QRadar Log Event Extended Format
Syslog (key=value)	Syslog strutturato
Word `.docx`	Preserva l'impaginazione; restituisce un `.docx` valido
Outlook `.pst`	Estrazione per messaggio; richiede `pst-utils` nell'immagine
PDF	Redazione con impaginazione preservata o estrazione di testo; richiede PyMuPDF

Robustezza a livello di byte: il parsing CSV rimuove i byte NUL incorporati (comuni negli export di eventi Windows) invece di fallire; i prefissi di trasporto usati dai wrapper Elastic/Kibana (`_source.`, `fields.`, `winlog.event_data.`, …) vengono rimossi prima del match sui campi, così il kit del prodotto interno si applica comunque.

---

8. Il motore di pseudonimizzazione

Il passaggio di mascheramento

`Anonymizer.process(text)` esegue una master regex sull'input che riconosce, in un ordine di precedenza fisso: URL, indirizzi e-mail, UPN, IPv4/IPv6, indirizzi MAC, pattern utente-nel-percorso (`C:\Users\jdoe\…`), FQDN e domini Windows. Ogni match viene instradato al builder di pseudonimi del suo tipo. Dopo il passaggio della regex, girano una serie di passaggi aggiuntivi:

1. **Hostname nudi** — token host non catturati dai pattern strutturati.
2. **Identità SharePoint** — percorsi `/personal/<chi>/`.
3. **Nomi di persona** — rilevamento nome/cognome tramite liste di serie e del tenant.
4. **Identità in prosa** — narrazioni `User mrossi logged on…` (vedi sotto).
5. **Scansione DLP** — credenziali, IBAN, codice fiscale, telefono, P.IVA, indirizzo, id cloud, parametri URL sensibili.
6. **Termini cliente** — i nomi cliente configurati, sempre mascherati per ultimi come sweep finale.

Pseudonimi deterministici e con chiave

Uno pseudonimo è un hash con chiave del valore:

```
pseudonimo = builder( HMAC(chiave_tenant, tipo, valore_normalizzato) )
```

Poiché la derivazione è deterministica e legata alla chiave del tenant, lo stesso valore produce sempre lo stesso token all'interno di un tenant, tenant diversi producono token diversi e la corrispondenza non è forzabile a forza bruta senza la chiave del tenant. Il valore originale è conservato cifrato nel vault, così il token è reversibile; i tipi irreversibili (nome cliente, segreto) non vengono mai messi nel vault.

Lo sweep dei valori noti al vault

Dopo il passaggio principale, LogMask può spazzare l'output alla ricerca dei valori già noti al vault e sostituirli con il loro pseudonimo canonico — è così che un host nominato in un messaggio in testo libero riceve lo stesso token assegnatogli nel suo campo dedicato. Lo sweep è adattivo nel costo: per un singolo evento su un vault grande cerca testo→vault tramite blind index (senza decifrare); per un export enorme su un vault piccolo

legge il vault una volta sola. Entrambe le strategie producono lo stesso output.

Nei contesti in linguaggio naturale (documenti, oggetti e corpi di e-mail) lo sweep gira in **modalità prosa**, che sostituisce solo gli originali che non possono essere parole comuni — quelli con una cifra o un separatore (`m.rossi`, `srv-01`, `DOMINIO\utente`) o composti da più parole (`Mario Rossi`). Questo impedisce che una parola comune finita nel vault per una vecchia classificazione errata (`SOC`, `Windows`, `Sicurezza`) corrompa il testo leggibile.

Guardie di correttezza

Gran parte del motore sono guardie che impediscono il mascheramento *sbagliato*, dannoso quanto una fuga perché distrugge gli IOC o corrompe il testo in silenzio:

- **Guardia hash-IOC** — una sequenza di ≥16 caratteri esadecimali è un indicatore (hash), mai un host; viene tenuta.
- **Guardia blob opaco** — un token adiacente a `+`, `/` o `=` è dentro un blob base64 (es. un `_id` di evento) e non viene mascherato, così gli identificatori univoci non vengono corrotti.
- **Guardia host-originale** — vengono spazzati solo i valori con la forma di un nome macchina; parole di prodotto (`Windows`, `Management`) e nomi di processo (`WmiPrvSE.exe`) no.
- **Guardia prosa-originale** — in linguaggio naturale vengono spazzati solo gli originali con forma di identificatore.
- **Identità nel testo** — le identità chiave/valore dentro i messaggi di evento Windows (`Account Name: mrossi`) e i JSON serializzati (`"SubjectUserName":"mrossi"`) vengono mascherate; i segnaposto dei log (`-`, `N/A`, `0x0`, `%%1833`, `localhost`, `SYSTEM`) no.
- **Controllo Luhn P.IVA** — un numero nudo di 11 cifre viene mascherato come partita IVA italiana solo se supera il controllo di Luhn; altrimenti è trattato come un record id e lasciato stare.
- **Guardia falsi positivi indirizzi** — gli indirizzi non etichettati richiedono un nome via con iniziale maiuscola più un numero civico, così `Potential RMM Tool Installation via Uncommon Process` e `traffic via proxy 8080` non vengono scambiati per indirizzi.

Idempotenza e reversibilità

Elaborare un output già mascherato non lo cambia: uno pseudonimo esistente viene riconosciuto e lasciato com'è. Il ripristino inverte ogni token reversibile; i token irreversibili (`CLIENT-…`, `secret-…`, `[ELIDED]`) restano, per scelta.

---

9. Riferimento delle forme dei token

Tipo	Input di esempio	Token di esempio	Reversibile
user	`mrossi`	`usr-4ozopszr`	sì
email	`a@acme.com`	`usr-6rwixc3e@osgwjo.masked`	sì
fqdn / host	`SRV-DC01.corp.local`	`host-ri6jxfsb.masked.local`	sì
ipv4 (interno)	`10.20.30.40`	`198.18.x.x`	sì
ipv4 (pubblico)	`8.8.8.8`	`198.19.x.x`	sì
ipv6	`fe80::1`	`fd00:…::1`	sì
mac	`00:1a:2b:3c:4d:5e`	`02:…` (locally-administered)	sì
windomain	`CORP`	`DOM-shphyawa`	sì
codice fiscale	`RSSMRA85T10A562S`	`cf-tyvu3zav7vpf`	sì

iban	`IT60X05428…`	`iban-dwchbizxmfec`	sì
telefono	`+39 335 1234567`	`tel-vzoftypx33b5`	sì
P.IVA	`00000017897`	`vat-y6orweoro2ds`	sì
indirizzo	`Via Roma 12`	`addr-6krvk6xmear5`	sì
persona	`Mario Rossi`	`person-… person-…`	sì
id cloud / UUID	`6f0c9a7e-…`	`cloud-g2xiyb2gkiq5`	sì
SID	`S-1-5-21-…`	`S-1-5-21-…` (sintetico)	sì
id opaco	(id base64 / hex)	`id-…`	sì
segreto	`password=Estate2024!`	`secret-…`	**no** (mai nel vault)
nome cliente	`Acme Spa`	`CLIENT-…`	**no** (mai nel vault)
eliso	(non classificato, Safe mode)	`[ELIDED]`	**no**

Gli IPv4 sintetici usano gli intervalli di benchmarking `198.18.0.0/15` (RFC 2544/6890) così da non poter essere confusi con indirizzi reali di produzione o privati; `198.18/16` marca le sorgenti interne e `198.19/16` quelle esterne. Gli IPv6 sintetici usano `fd00::/8` (unique-local). I MAC sintetici usano il prefisso locally-administered `02:…`.

---

10. Kit vendor

Un kit è un file YAML che mappa i nomi campo di un vendor a tipi e azioni. Il rilevamento avviene tramite campi fingerprint più suggerimenti d'intestazione; vince il kit col punteggio più alto, soggetto a una confidenza minima.

Kit di serie (21): Acronis, AWS CloudTrail, Bitdefender, Cisco Secure Endpoint, Cortex (Palo Alto XDR / XSIAM / XSOAR), CrowdStrike, Darktrace, Elastic ECS, Exabeam (New-Scale CIM 2.0 / Advanced Analytics / Data Lake), Fortinet, Microsoft Defender, Microsoft Entra, Microsoft Sentinel, Okta, Proofpoint, SentinelOne, Sophos, Splunk CIM, Trend Vision One, Wazuh, Zscaler.

Struttura delle regole. Ogni regola ha un `pattern` (una regex confrontata col nome campo) più un'`action` e, per `mask`, un `kind`:

```
- pattern: ^(dest|src)_user_(sid|dn|ou|entity_id)$
  action: mask
  kind: user
- pattern: ^action_evtlog_event_id$
  action: keep
- pattern: ^(message|raw_log|description)$
  action: text
```

Le regole sono valutate dall'alto verso il basso; quelle specifiche devono precedere i catch-all generici (un campo come `action_evtlog_event_id` va tenuto *prima* che la regola generica `.*_id$` lo maschererebbe).

Kit utente. I file in `data/kits/*.yaml` vengono caricati sopra quelli di serie con hot reload e hanno priorità. Usa il kit studio per aprire un kit di serie come copia, modificarlo, validarlo ed eseguire un dry-run su un'intestazione prima di salvare.

La copertura è misurata, non presunta. Il kit Exabeam, per esempio, è costruito sui 1165 nomi di campo pubblicati nel repository ufficiale `ExabeamLabs/CIMLibrary`; un test di regressione verifica che nessun campo il cui nome denota un'identità resti leggibile, mentre i campi operativi restano leggibili.

Nessun kit può tenere in chiaro i campi sconosciuti. Una regola di kit è taggata `vendor:` e il Safe mode di proposito non ri-elide i campi classificati da un kit — quindi un catch-all `.* → keep` in fondo a un kit terrebbe leggibile *ogni* campo non riconosciuto, annullando il fail-closed. Un test di regressione sull'intera classe vieta un `.* → keep` (e un `.* → mask` con un kind che non maschera) in qualsiasi kit. Dove un kit ha bisogno di un catch-all per un vendor con moltissimi campi, usa `.* → text`, che processa i valori sconosciuti come testo libero — mascherando ogni identità al loro interno (IP, e-mail, host, `DOMINIO\utente`, nome persona o cliente) e lasciando leggibile il contenuto operativo. Ogni kit deve classificare utenti e IP, e anche gli host tranne i vendor senza un concetto di host (Proofpoint, AWS CloudTrail, Okta, Entra), elencati esplicitamente così un'eccezione è una scelta e non una svista.

---

11. Categorie DLP e Safe mode

Indipendentemente dai kit vendor, una scansione DLP trova i valori sensibili ad alta confidenza ovunque nell'output — anche dentro il testo libero. Ogni categoria ha un'azione predefinita e può essere sovrascritta per richiesta.

Categoria	Tipo	Default
`credentials`	secret	redact
`private_key`	secret	redact
`tax_id`	taxid	pseudonymize
`iban`	iban	pseudonymize
`phone`	phone	pseudonymize
`person_name`	person	pseudonymize
`address`	address	pseudonymize
`vat_id`	vat	pseudonymize
`cloud_id`	cloud	pseudonymize
`sensitive_url`	secret	redact

Azioni: `pseudonymize` (token reversibile), `redact` (`[ELIDED]`, irreversibile), `block` (fa fallire l'output), `keep` (lascia in chiaro).

Safe mode è l'interruttore fail-closed. Con Safe mode attiva, qualsiasi campo popolato che nessun kit e nessuna euristica ha saputo classificare viene eliso. Il ragionamento è che un campo non classificato è esattamente il posto dove si nasconde un identificatore inatteso; lasciarlo passare in chiaro sarebbe l'unico punto da cui una fuga sfugge.

Documenti e posta non elidono mai. Sui percorsi `.docx`, `.pst` e `.pdf`, elidere un campo è un danno netto — il file restituito perde testo e il ripristino non può ricostruirlo. Su quei percorsi ogni valore sensibile diventa uno pseudonimo invece di `[ELIDED]`. I segreti sono l'unica eccezione su quei percorsi: diventano un token deterministico `secret-…` mai scritto nel vault, così il documento resta leggibile ma il segreto non è recuperabile e lo strumento non diventa mai un deposito di password dei clienti.

Override dei campi non tracciati

I campi che nessun kit classifica vengono elisi in Safe mode. Per cambiare il trattamento di un campo specifico senza scrivere una regola di kit, il report mostra ogni campo non tracciato con una tendina a tre scelte e un pulsante **Salva config campi mancanti**:

- **mantieni** — lascia il valore leggibile (metadati operativi, IOC);

- **pseudonimizza** — token reversibile `id-…` (sempre kind generico `opaque`, che maschera qualsiasi valore; un kind tipizzato come `ipv4` NON viene offerto di proposito, perché restituirebbe invariati i valori non conformi — una fuga);
- **elidi** — `[ELIDED]`.

Le scelte sono salvate globalmente per nome campo in `data/field_overrides.json` (hot reload) e valgono per ogni formato — CSV, JSON/NDJSON, CEF, LEEF, syslog — perché risolte prima del rilevamento vendor. Sono indipendenti dal vendor rilevato, dato che i campi non tracciati compaiono di solito proprio quando nessun kit ha fatto match. Gli override vincono su kit, catalogo ed euristiche, ma un campo senza override continua a essere eliso in Safe mode: il fail-closed è preservato. Il salvataggio richiede il permesso `admin`, come le altre modifiche di configurazione. Dalla 0.27.0 `redact` è anche un'azione valida nei kit, quindi un kit utente può forzare l'elisione di un campo.

---

12. Riferimento API

Tutti gli endpoint sono sotto `/api`. La sessione è un cookie; le richieste che modificano lo stato richiedono il token CSRF (double-submit: il valore del cookie `logmask_csrf` ripetuto nell'header `X-CSRF-Token`). Ogni richiesta è autorizzata per ruolo e, dove pertinente, per tenant.

Autenticazione

Metodo	Percorso	Permesso	Scopo
POST	`/api/login`	—	Login; restituisce i cookie di sessione + CSRF
GET	`/api/me`	sessione	Utente corrente, ruoli, capacità
POST	`/api/logout`	sessione	Termina la sessione
POST	`/api/change-password`	sessione	Cambia la propria password

Anonimizza / ripristina

Metodo	Percorso	Permesso	Scopo
POST	`/api/anonymize`	anonymize	Input testo / CSV / strutturato (corpo JSON)
POST	`/api/anonymize-docx`	anonymize	Documento Word (multipart)
POST	`/api/anonymize-pst`	anonymize	Archivio Outlook (multipart)
POST	`/api/anonymize-pdf`	anonymize	PDF (multipart; `output=pdf|text`)
POST	`/api/deanonymize`	reverse	Ripristina testo / strutturato / CSV
POST	`/api/deanonymize-docx`	reverse	Ripristina un `.docx` mascherato
POST	`/api/deanonymize-pdf`	reverse	Ripristina un PDF mascherato

Kit e policy

Metodo	Percorso	Permesso	Scopo
GET	`/api/vendor-kits`	sessione	Elenca i kit installati
POST	`/api/kit-dry-run`	sessione	Classifica un'intestazione senza elaborare valori
GET	`/api/kits/files`	admin	Elenca i file kit utente
GET	`/api/kits/files/{name}`	admin	Legge un kit utente
PUT	`/api/kits/files/{name}`	admin	Crea/aggiorna un kit utente (validato)

DELETE	`/api/kits/files/{name}`	admin	Elimina un kit utente
POST	`/api/kits/validate`	admin	Valida uno YAML di kit
GET	`/api/kits/bundled/{kit_id}`	admin	Legge un kit di serie
GET	`/api/dlp-categories`	sessione	Catalogo delle categorie DLP
GET	`/api/field-overrides`	sessione	Override per campo attuali
POST	`/api/field-overrides`	admin	Salva gli override per campo (keep/mask/redact)
GET	`/api/workflow-profiles`	sessione	Preset di lavoro

Reportistica e amministrazione

Metodo	Percorso	Permesso	Scopo
GET	`/api/tenants`	sessione	Tenant a cui l'utente può accedere
GET	`/api/fields`	reports	Statistiche a livello di campo
GET	`/api/stats`	reports	Statistiche del vault
GET	`/api/admin/users`	admin	Elenca gli utenti
POST	`/api/admin/users`	admin	Crea un utente
POST	`/api/admin/vault/reset`	admin	Archivia-e-azzera il vault di un tenant
POST	`/api/admin/secret/reset`	admin	Rigenera la master key (archivia chiave + tutti i vault)
GET	`/api/admin/audit`	audit	Log di audit

Esempio — anonimizza testo

```
curl -X POST http://localhost:8090/api/anonymize \
  -H "Content-Type: application/json" \
  -H "X-CSRF-Token: <csrf>" \
  --cookie "logmask_session=<session>; logmask_csrf=<csrf>" \
  -d '{"tenant":"acme","text":"user=mrossi src=10.20.30.40","format":"auto",
       "safe_mode":true,"ip_mode":"all","url_mode":"all"}'
```

Gli errori sono restituiti come JSON con un campo `detail` e, per gli errori imprevisti, un `error_type`; la traccia completa resta nei log del container e non viene mai esposta.

---

13. Modello di sicurezza

Autenticazione e sessioni

- **Le password** sono hashate con Argon2 (`time_cost=3`, `memory_cost=64 MiB`, `parallelism=2`); lunghezza minima 12; lo username non può comparire nella password.
- **Le sessioni** sono cookie marcati `HttpOnly` e `SameSite=strict`; imposta `LOGMASK_COOKIE_SECURE=true` dietro HTTPS. Vengono applicati timeout di inattività e durata assoluta.
- **Il CSRF** usa double-submit: un cookie `logmask_csrf` non-`HttpOnly` deve essere ripetuto nell'header `X-CSRF-Token` a ogni richiesta che modifica lo stato.
- **Rate limiting sul login**: dopo `LOGMASK_LOGIN_MAX_FAILURES` fallimenti nella finestra, l'account è bloccato per la durata della finestra.
- **L'admin di bootstrap** deve cambiare la password al primo accesso prima di qualsiasi altra azione.

Ruoli (RBAC)

Ruolo	Permessi
`operator`	anonymize
`analyst`	anonymize, reports
`reverser`	anonymize, reports, reverse
`admin`	anonymize, reports, reverse, admin, audit

Il ripristino (`reverse`) è un privilegio distinto e superiore e ogni ripristino è scritto nel log di audit con utente, tenant, IP ed esito.

Isolamento dei tenant

Gli identificatori di tenant sono validati da regex e canonicalizzati in una forma sicura per i percorsi (nessun traversal `./..`), poi autorizzati contro l'ambito di tenant dell'utente. Vault e chiavi derivate sono per tenant; un utente può operare solo sui tenant che gli sono concessi.

Crittografia

- Una chiave master di 32 byte con permessi `0600`.
- Chiavi per tenant derivate tramite HMAC.
- Valori originali cifrati con AES-GCM, un nonce casuale di 12 byte per riga.
- I blind index (hash con chiave) permettono allo sweep di cercare i valori senza decifrare il vault.

Robustezza sugli input

- **Limiti di dimensione** per richiesta e file (413 al superamento).
- **Guardia zip-bomb** `.docx`: un tetto sulla dimensione decompressa con doppio controllo (dimensioni dichiarate, poi un budget di lettura), perché l'indice zip può mentire. Verificato: un documento da 200 KB che si espande a centinaia di MB viene respinto in millisecondi.
- **Tetto sull'estrazione** `.pst`: la dimensione estratta è limitata; `readpst` gira con un timeout e stdin chiuso, così un archivio corrotto o protetto da password non può bloccare il worker.
- **PDF**: tetto sul numero di pagine; i PDF cifrati/corrotti vengono respinti con un messaggio chiaro; il PDF prodotto viene riletto e si verifica che nessun valore originale sia ancora estraibile prima della consegna.
- **L'espansione di entità XML** (billion-laughs) è bloccata dal parser.
- **I byte NUL nei CSV** vengono rimossi invece di far crashare il parser.
- **Header Content-Security-Policy** e `Cache-Control: no-store` impostati; tutto il DOM dinamico nell'interfaccia è inserito via `textContent`/escaping, non HTML grezzo.

Considerazioni residue (responsabilità dell'operatore)

- Il bind predefinito è `0.0.0.0` (raggiungibile da LAN); imposta `LOGMASK_BIND=127.0.0.1` per uso solo host.
- Il container gira come root; i tetti sugli input di cui sopra mitigano ma non eliminano questo aspetto.
- `readpst` è codice C di terze parti; il timeout e il tetto sulla dimensione ne limitano l'impatto, non le vulnerabilità interne.
- `client_terms.txt` e `person_terms.txt` contengono valori reali e identificativi e vanno protetti di conseguenza.

14. Operazioni e risoluzione problemi

Reset del vault

`Reset vault` (solo admin) archivia il vault del tenant con un timestamp invece di eliminarlo. Poiché il vault è l'unica cosa che rende reversibili gli export passati, azzerarlo significa perdere la corrispondenza per tutto ciò che è già stato condiviso; l'archiviazione consente il recupero se l'azzeramento è stato un errore. La cancellazione definitiva resta una scelta manuale e consapevole.

Reset secret (master key)

`Reset secret` (solo admin) rigenera la master key — il secret da cui derivano **tutti** i token. Un pop-up riassume il rischio e richiede di digitare la parola `RESET`. L'effetto è globale e distruttivo per la reversibilità:

- ogni token futuro cambia (lo stesso valore produce uno pseudonimo diverso, `secret-…` compresi);
- **ogni vault esistente, di ogni tenant, smette di essere reversibile**, perché cifrato con la chiave precedente.

Come per il reset del vault, nulla viene cancellato: la vecchia chiave e tutti i vault vengono **archiviati** con un timestamp (`master-prereset-…key`, `vault-prereset-…db`) e sono recuperabili **solo insieme** — rimettendo entrambi al loro posto si torna indietro. Utenti e sessioni non dipendono dalla master key, quindi nessuno viene disconnesso, e il cambio ha effetto subito senza riavvio.

Usalo per ottenere un secret nuovo e scollegato — per esempio dopo aver copiato la cartella su un'altra macchina, se vuoi deliberatamente che il nuovo deployment produca token diversi. Al contrario, per mantenere la continuità tra macchine, copia `data/master.key` (e i vault) invece di fare il reset.

Log di audit

Ogni operazione di anonimizzazione, ripristino, salvataggio override e reset del secret è registrata con azione, utente, tenant, IP ed esito (più i conteggi e, per i fallimenti, il tipo di eccezione — mai il contenuto del log). Usalo per verificare chi ha ripristinato o resettato cosa e quando.

Situazioni comuni

Sintomo	Causa probabile / azione
`PST anonymization failed: Failed to fetch`	Il `.pst` era ancora aperto in Outlook; il file è cambiato durante l'upload. Chiudi Outlook o lavora su una copia. LogMask legge il file in una copia stabile prima di inviarlo.
`il PDF è protetto da password`	Rimuovi la protezione del PDF (aprilo e salvane una copia senza password) prima di anonimizzarlo.
Un campo esce in chiaro	Non è stato classificato da alcun kit; attiva Safe mode, oppure aggiungi una regola in un kit utente / una voce in `keep_fields`, oppure esegui un dry-run del kit per vedere la classificazione.
È tutto `[ELIDED]`	Nessun kit vendor ha fatto match e Safe mode ha eliso i campi ignoti; forza il kit corretto o aggiungi un kit utente.
Il ripristino lascia alcuni valori come pseudonimi	Quei token sono irreversibili per scelta (`CLIENT-…`, `secret-…`) o appartengono al vault di un altro tenant.
È diventato lento	Una release precedente aveva uno sweep dipendente dalla dimensione del vault; le release attuali sono indipendenti dal vault. Token singoli molto grandi erano un vettore ReDoS corretto nella 0.26.2.

Aggiornamento

Copia la nuova release sopra la cartella del progetto (o estrai lo ZIP di release), poi `docker compose down && docker compose build --no-cache && docker compose up -d`. I vault e la configurazione nella cartella dati sono preservati tra gli aggiornamenti.

---

15. Considerazioni normative (UE / Italia, uso dell'AI)

> **Questa sezione è informativa, non è un parere legale.** Riassume come LogMask si colloca nel quadro normativo europeo e italiano al 2026, così da permetterti una valutazione informata con il tuo DPO o un legale. Non è un'opinione di conformità e gli autori di LogMask non sono i tuoi avvocati.

15.1 Il fatto giuridico centrale: pseudonimizzazione non è anonimizzazione

Ai sensi del GDPR (Regolamento (UE) 2016/679), la **pseudonimizzazione** è definita all'articolo 4, punto 5, come il trattamento dei dati personali in modo tale che non possano più essere attribuiti a un interessato senza informazioni aggiuntive conservate separatamente. Fondamentale: **il Considerando 26 stabilisce che i dati pseudonimizzati, che potrebbero essere attribuiti a una persona fisica mediante l'uso di informazioni aggiuntive, restano dati personali.** L'anonimizzazione, al contrario, è irreversibile e — sempre per il Considerando 26 — esce dal campo di applicazione del GDPR.

LogMask esegue pseudonimizzazione. Di conseguenza:

- **Un export mascherato da LogMask è, in generale, ancora un dato personale** ai sensi del GDPR, perché il vault più la chiave del tenant sono esattamente quelle "informazioni aggiuntive" che consentono la re-identificazione. Trattarlo (conservarlo, condividerlo, inviarlo a un servizio AI) richiede comunque una base giuridica e attiva comunque gli obblighi del GDPR.
- **Il vault e la chiave master sono le "informazioni aggiuntive"** che l'articolo 4, punto 5, richiede siano "conservate separatamente e soggette a misure tecniche e organizzative". La cifratura AES-GCM per tenant di LogMask, la chiave master a `0600`, l'RBAC e il log di audit sono il tipo di misure che l'articolo contempla — ma la *separazione* (tenere il vault fuori dalle mani del destinatario) è una responsabilità operativa: non consegnare mai il vault insieme a un export mascherato.
- **Le trasformazioni irreversibili si comportano diversamente.** I valori che LogMask non memorizza mai — nomi cliente (`CLIENT-…`), segreti (`secret-…`) e campi elisi (`[ELIDED]`) — non sono reversibili nemmeno da chi possiede il vault, e per quei valori specifici l'export si avvicina all'anonimizzazione. Il resto resta pseudonimo.

Conclusione pratica: LogMask è un **controllo di minimizzazione e riduzione del rischio**, non una bacchetta magica che sottrae i dati al campo di applicazione del GDPR. Riduce in modo sostanziale i dati personali esposti a un destinatario e sostiene il principio di minimizzazione (art. 5, par. 1, lett. c) e l'obbligo di sicurezza del trattamento (art. 32), che è precisamente il suo valore.

15.2 Inviare log a un servizio AI

Il caso d'uso specifico a cui mira LogMask — mascherare un log prima di inviarlo a un LLM o assistente AI esterno — si colloca all'interno di due regimi che si sovrappongono.

GDPR. Inviare un log a un servizio AI di terze parti è un'operazione di trattamento e, se il fornitore è fuori dall'UE/SEE, potenzialmente un trasferimento. Richiede una base giuridica, un accordo sul trattamento dei dati con il fornitore e l'aderenza al principio di minimizzazione. Il **Garante per la protezione dei dati personali** ha ripetutamente sottolineato la difficoltà di applicare la minimizzazione ai servizi di AI generativa e ha adottato provvedimenti in quest'area (in particolare il caso ChatGPT, chiuso a dicembre 2024 con una sanzione da 15 milioni di euro; si noti che un tribunale italiano ha successivamente annullato il relativo provvedimento, quindi la giurisprudenza è ancora in evoluzione). Pseudonimizzare il log prima che esca dal tuo controllo è un modo

diretto e difendibile di onorare la minimizzazione: il servizio AI vede `usr-4ozopszr`, non `mrossi`.

Regolamento UE sull'IA — AI Act (Regolamento (UE) 2024/1689). L'AI Act si applica progressivamente. Gli obblighi per i modelli di IA per finalità generali (GPAI) si applicano dal **2 agosto 2025**; una tappa rilevante cade il **2 agosto 2026**, quando entrano in vigore gli obblighi di trasparenza (art. 50, es. dichiarare che un utente sta interagendo con un sistema di IA) e il regime sanzionatorio per i GPAI (ammende fino al 3% del fatturato mondiale annuo o 15 milioni di euro, se superiore). Le tempistiche per alcuni sistemi ad alto rischio sono state successivamente differite (a dicembre 2027 e agosto 2028) dal "Digital Omnibus". LogMask di per sé non è un sistema di IA e non è regolato dall'AI Act; la sua rilevanza è di essere un controllo che puoi porre *a monte* di un sistema di IA per ridurre i dati personali che quel sistema tratta.

15.3 La legge nazionale italiana sull'IA (Legge 132/2025)

L'Italia ha adottato la sua prima legge organica sull'IA, la **Legge 23 settembre 2025, n. 132** ("Disposizioni e deleghe al Governo in materia di intelligenza artificiale"), pubblicata in Gazzetta Ufficiale il 25 settembre 2025 ed **in vigore dal 10 ottobre 2025**. È espressamente coordinata con l'AI Act europeo e aggiunge indirizzi di livello nazionale. Individua due autorità nazionali: l'**Agenzia per l'Italia Digitale (AgID)** per la promozione e lo sviluppo, e l'**Agenzia per la Cybersicurezza Nazionale (ACN)** per la vigilanza e la sicurezza. Per un'organizzazione che usa l'IA su dati che possono contenere informazioni personali o sensibili sul piano della sicurezza, la legge rafforza i principi di trasparenza, sorveglianza umana e sicurezza che un controllo di pre-trattamento come LogMask aiuta a rendere operativi.

15.4 Cosa LogMask ti dà e cosa non ti dà

Ti aiuta a: minimizzare i dati personali e identificativi del cliente esposti quando un log viene condiviso o inviato a un servizio AI; mantenere una corrispondenza reversibile e verificabile sotto il tuo controllo; dimostrare una misura tecnica concreta verso la minimizzazione (art. 5, par. 1, lett. c) e la sicurezza (art. 32); e rimuovere in modo irreversibile gli elementi più sensibili (nomi cliente, segreti).

Non fa, da solo: rendere "anonimo" un export mascherato né sottrarlo al campo del GDPR (il vault lo rende re-identificabile); fornire una base giuridica per il trattamento o il trasferimento; sostituire un accordo sul trattamento con un fornitore AI; né esimerti da una DPIA dove richiesta. Queste restano decisioni organizzative.

Indicazioni operative che ne discendono:

1. **Non consegnare mai il vault insieme all'export.** La separazione è ciò che rende significativa la pseudonimizzazione ai sensi dell'art. 4, punto 5.
2. **Proteggi il vault e la chiave master** come gli asset sensibili che sono — sono la chiave di re-identificazione.
3. **Preferisci le impostazioni più protettive** quando il destinatario è esterno, soprattutto un servizio AI: tutti gli IP e gli URL mascherati (i default della 0.26.1), Safe mode attiva, segreti e PII minimizzati. Il profilo di lavoro "Analisi AI esterna" codifica questo.
4. **Tratta** `client_terms.txt` / `person_terms.txt` come configurazione riservata.
5. **Conserva il log di audit** come prova di chi ha ripristinato cosa e quando.

---

16. Estendere LogMask

Aggiungere o sovrascrivere un kit vendor

Crea un file YAML in `data/kits/`. Viene ricaricato a caldo e ha priorità sul kit di serie con lo stesso id. Usa il kit studio per partire da una copia di un kit di serie, poi valida ed esegui un dry-run prima di salvare. Metti le

regole specifiche prima dei catch-all generici, e preferisci `keep` per i campi operativi e gli IOC, `mask` (con un `kind`) per le identità e `text` per i campi in testo libero.

Aggiungere un rilevatore DLP

Le categorie DLP stanno in `dlp.py` con etichetta, descrizione, azione predefinita e tipo. Un nuovo rilevatore aggiunge una regex allo scanner dei residui e, dove il valore debba essere reversibile, un corrispondente builder di pseudonimi in `logmask.py`. Le nuove categorie compaiono automaticamente nel pannello DLP dell'interfaccia.

Regolare il rilevamento di persone / host / cliente

Popola i file dati: `person_terms.txt` per i dipendenti reali (mascherati anche come token nudo), `host_terms.txt` per i glob di nomi host del tenant, `tenant_networks.txt` per i CIDR pubblici che identificano il cliente, `client_terms.txt` per i nomi cliente. Le liste di serie in `data/persons/` alimentano il rilevamento generico dei nomi.

Disciplina dei test

Ogni cambiamento di comportamento è accompagnato da un test di regressione; la suite è la specifica del comportamento voluto, in particolare per le guardie (un mascheramento troppo aggressivo è trattato come un bug alla pari di una fuga). Esegui l'intera suite prima di costruire una release.

LogMask 0.27.7 — documentazione tecnica. Per lo storico delle modifiche vedi `CHANGELOG.md`.